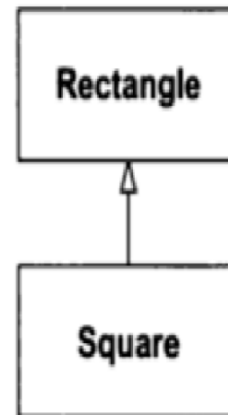


Liskov Substitution Principle

Square class inherits from Rectangle class.



```
class Rectangle {
  constructor(width, height) {
    this.width = width || 0;
    this.height = height || 0;
  }

  getWidth() {
    return this.width;
  }

  setWidth(width) {
    this.width = width;
  }

  getHeight() {
    return this.height;
  }

  setHeight(height) {
    this.height = height;
  }

  getArea() {
    return this.width * this.height;
  }
}
```



```
class Square extends Rectangle {
  constructor(size) {
    super(size, size);
  }

  setWidth(w) {
    this.width = this.height = w;
  }

  setHeight(h) {
    this.width = this.height = h;
  }
}
```



```
function getAreaTest(r) {
  const w = r.getWidth();
  r.setHeight(10);

  console.log(`Expected area =
    ${w * 10}, got = ${r.getArea()}`);
}

const rc = new Rectangle(2, 3);
getAreaTest(rc);          // Expected area = 20, got = 20

const sq = new Square(4);
getAreaTest(sq);          // Expected area = 40, got = 100
```

So when the test does:

1. `w = sq.getWidth();` → returns 4
2. `sq.setHeight(10);` → width ALSO becomes 10
3. Expected area = $4 * 10 = 40$
4. Actual area = $10 * 10 = 100$

Rectangle requires **independent width and height**

Square requires **width === height**

Those two requirements **cannot both be true**, so inheritance is invalid.

👉 **This breaks Liskov Substitution Principle because:**

A Square **cannot behave like** a Rectangle when someone tries to change width or height independently.

Therefore it cannot safely substitute Rectangle in all contexts.

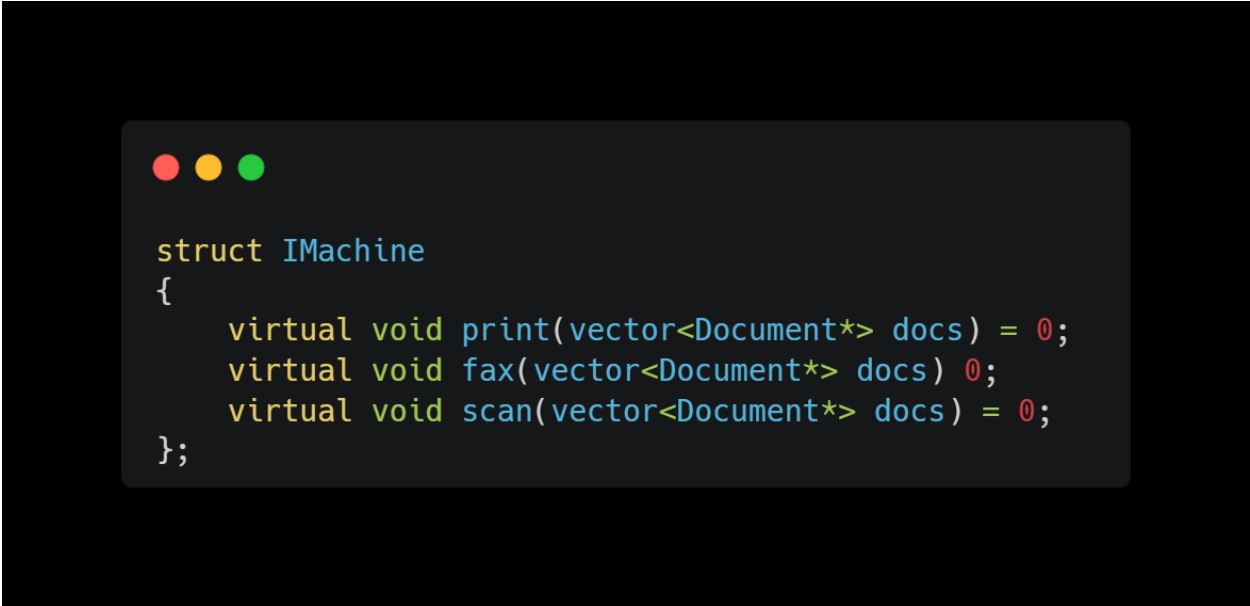
Interface Segregation Principle

Interface-Segregation Principle states:

“Clients should not be forced to implement methods that they do not need.”

Example: Suppose you want to represent a multifunction device that can print, scan and also fax documents.

You can define an interface for it like that:



```
struct IMachine
{
    virtual void print(vector<Document*> docs) = 0;
    virtual void fax(vector<Document*> docs) = 0;
    virtual void scan(vector<Document*> docs) = 0;
};
```

What is the problem with that ?

- If there is some device that implements this interface but wants only to do scanning but not printing or sending faxes.

Better design

Define separate interfaces for each task.

- A concrete class will implement only interfaces for tasks it can handle.
- A concrete class can implement as many services as it needs.



```
struct IFax
{
    virtual void fax(vector<Document*> docs) = 0;
};

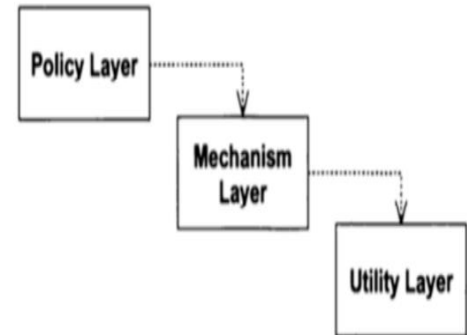
struct IPrinter
{
    virtual void print(vector<Document*> docs) = 0;
};

struct IScanner
{
    virtual void scan(vector<Document*> docs) = 0;
};
```

Dependency Inversion Principle

Dependency Inversion Principle states:

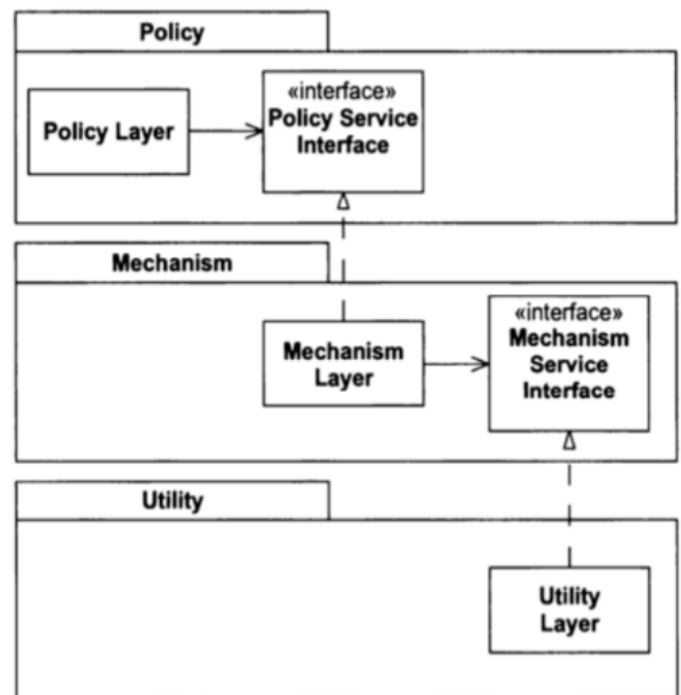
“High level modules should not depend on low level modules. Both should depend on abstractions”



- **Policy Layer** uses a lower level **Mechanism Layer**.
- **Mechanism Layer** depends on a lower **Utility Layer**.
- Dependency is **transitive**, Policy Layer depends on changes from both mechanism and utility layers.

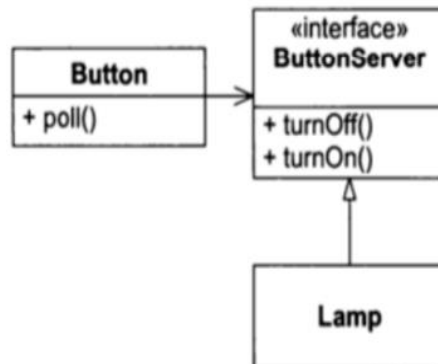
Re-design:

- Upper layer define an interface for services they need.
- Lower layers are realized from these abstract interfaces.
- Each higher level layer uses next lower layer through the abstract interface.
- Higher layer do not depend on lower layers, but instead lower layers depend on abstract services declared in upper layers.
- This also breaks the transitive dependency between “**Policy Layer**” and “**Utility Layer**”.



Pros of DIP redesign:

- Higher level (Policy Layer) module is unaffected by changes in Mechanism Layer or UtilityLayer.
- PolicyLayer can be reused in any context that defines low-level modules that conform to Policy Service Interface. Therefore, the structure is more flexible.



```
#include <iostream>

class IDevice {
public:
    virtual void TurnOn() = 0;
    virtual void TurnOff() = 0;
    virtual ~IDevice() = default;
};

// -----Lamp-----

class Lamp : public IDevice {
public:
    void TurnOn() override {
        std::cout << "Turning lamp on!" << std::endl;
    }

    void TurnOff() override {
        std::cout << "Turning lamp off!" <<
std::endl;
    }
};
```

```

// -----Button-----

class Button {
private:
    IDevice* device;
    bool pressed = false;

public:
    Button(IDevice* device) {
        this->device = device;
    }

    void Press() {
        if (pressed) {
            device->TurnOn();
        } else {
            device->TurnOff();
        }
        pressed = !pressed;
    }
};

```

```

// -----Engine-----

class Engine : public IDevice {
public:
    void TurnOn() override {
        std::cout << "---- Engine is running" << std::endl;
    }

    void TurnOff() override {
        std::cout << "---- Engine is stopped." << std::endl;
    }
};

```